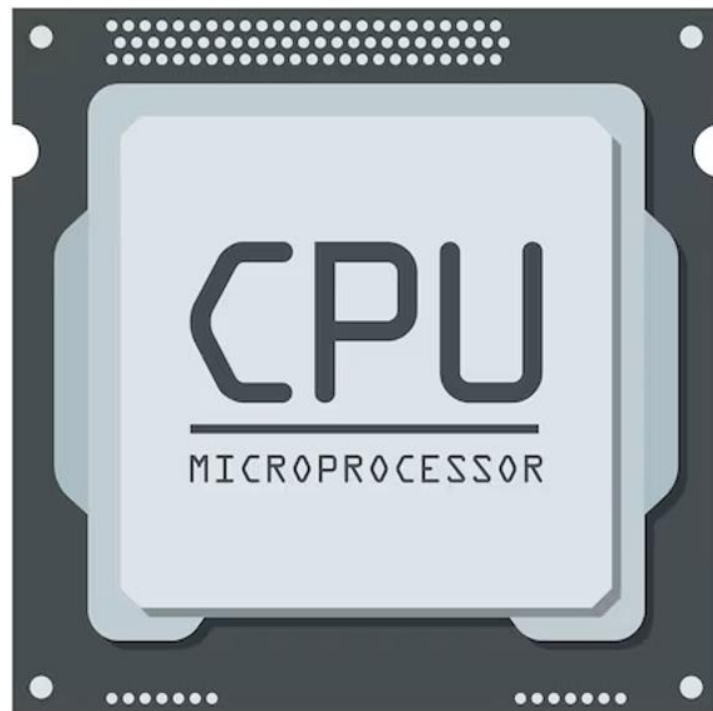


EL KATTOUFI Youssef

Année 2025 – 2026

Processeur Monocycle et Multicycle



Etablissement scolaire : Polytech Sorbonne

Formation : Electronique et Informatique

Matières : VHDL comportemental

Enseignant : Yann DOUZE

Sommaire

Remerciements.....	5
Résumé	6
I. Présentation du projet	7
II. Conception et Architecture Fonctionnelle	8
A. Unité Arithmétique et Logique (ALU).....	9
1. Explication du code de l'ALU.....	9
2. Explication du banc de test de l'ALU	10
3. Résultat de la simulation de l'ALU	10
B. Banc de registre.....	11
1. Explication du code du banc de registre	11
2. Explication du banc de test du banc de registre.....	11
3. Résultat de la simulation du banc de registre	12
C. Le multiplexeur 2 vers 1	13
1. Explication du code du multiplexeur 2 vers 1	13
2. Explication du banc de test du multiplexeur 2 vers 1	14
3. Résultat de la simulation du multiplexeur 2 vers 1	14
D. Extension de signe	16
1. Explication du code de l'extension du signe	16
2. Explication du banc de test du multiplexeur 2 vers 1	16
3. Résultat de la simulation du multiplexeur 2 vers 1	17
E. Mémoire de données	18
1. Explication du code sur la mémoire de données	18
2. Explication du banc de test sur la mémoire de données.....	18
3. Résultat de la simulation de la mémoire de donnée	19
F. Assemblage Unité de Traitement	20
1. Explication du banc de test du datapath.....	21
2. Résultat de la simulation du datapath	22

G.	Registre de commande 32bits	22
H.	Unité de gestion d'instruction	23
1.	Explication du code de l'Unité de Gestion d'Instruction	23
2.	Explication du banc de test de l'Unité de Gestion d'Instruction	24
3.	Résultat de la simulation de l'Unité de Gestion d'Instruction	24
I.	Décodeur	26
1.	Explication du code du décodeur	26
a)	Valeur des commandes	27
2.	Explication du banc de test de l'Unité de Gestion d'Instruction	27
3.	Résultat de la simulation de l'Unité de Gestion d'Instruction	28
III.	Validation du processeur monocycle	29
A.	Explication du banc de test.....	29
B.	Programme exécuté par le processeur monocycle	30
C.	Analyse de la simulation	30
IV.	Quartus prime	33
A.	Preuve de fonctionnement sur FPGA	34
Annexe.....		36
Tables des figures		37

Remerciements

Je tiens tout d'abord à remercier chaleureusement Monsieur Yann DOUZE qui nous a encadré tout au long du semestre 6. Son accompagnement, ses conseils et son enseignement m'a permis de mener à bien ce projet, tout en nous apportant le socle de connaissances nécessaires en vhdl comportemental.

Résumé

Au sein de la formation d'ingénieur en Électronique et Informatique à Polytech Sorbonne, nous avons suivi l'enseignement de VHDL comportemental, qui nous a permis d'acquérir des connaissances fondamentales sur l'architecture et le fonctionnement des processeurs monocycle et multicycle.

Cette matière nous a notamment initiés aux différents composants internes d'un processeur, tels que l'ALU, le décodeur d'instructions, l'unité de contrôle, la mémoire de données et les registres. Nous avons ainsi pu mieux comprendre les mécanismes qui permettent l'exécution des instructions et la gestion des flux de données au sein d'un système numérique.

Cet enseignement m'a également sensibilisé à la complexité de la conception et de l'implémentation des processeurs. J'ai notamment pris conscience de l'importance des phases de simulation et de validation du système avant sa synthèse sur FPGA. En effet, des simulations bien faites permettent de détecter les erreurs en amont, de réduire le temps de développement et d'éviter de longues phases de compilation sous Quartus Prime.

Enfin, cette expérience a mis en évidence l'importance de la modélisation et de l'anticipation du comportement d'un système numérique, afin de vérifier son bon fonctionnement avant toute réalisation matérielle.

I. Présentation du projet

Ce mini-projet a pour objectif la conception, la simulation et l'implémentation sur FPGA d'un processeur inspiré de l'architecture ARM7TDMI, en langage VHDL sur la carte DE10-Lite (voir figure ci-dessous). Le but est de reproduire le fonctionnement interne d'un processeur capable d'exécuter un jeu d'instructions de type ARM, en mettant en œuvre les principaux blocs d'un calculateur numérique.

Deux architectures ont été étudiées : le processeur monocycle, où chaque instruction est exécutée en un seul cycle d'horloge, et le processeur multicycle, où l'exécution est répartie sur plusieurs cycles afin d'optimiser l'utilisation des ressources matérielles.

À travers ce travail, nous avons pu analyser le chemin de données et le chemin de contrôle du processeur, comprendre la gestion des instructions, des accès mémoire et des opérations arithmétiques, puis valider chaque module par simulation logique avant de l'exporter sur la carte DE10-Lite. Des bancs de test et des scripts de simulation automatisés sous ModelSim ont été développés afin de vérifier que le comportement était le bon.

Une fois la conception validée, le processeur a été synthétisé et déployé sur la carte FPGA à l'aide de l'environnement Intel Quartus Prime, incluant l'assignation des broches.

Il met également en évidence l'importance des phases de simulation et de vérification en amont, qui permettent de détecter les erreurs dès la conception, de réduire le temps de développement et d'éviter des pertes de temps lors de l'implémentation sur FPGA.

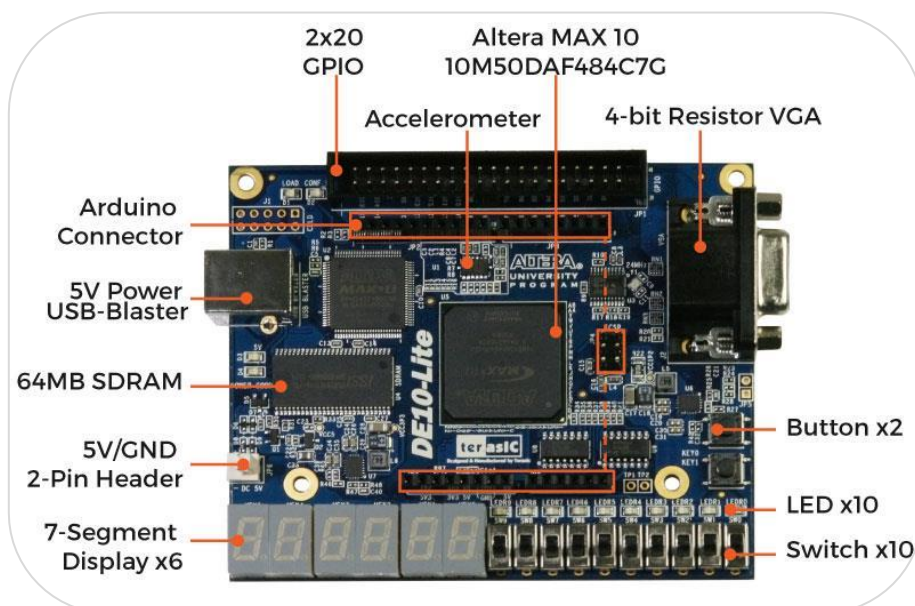


Figure 1 : Carte DE10-Lite

Remarque :

Faute d'une bonne gestion de mon temps personnels, j'ai préféré peaufiner le monocycle et ne pas commencer la partie multicycle

II. Conception et Architecture Fonctionnelle

Dans ce compte-rendu, nous présentons la démarche complète de conception des processeurs monocycle. Nous détaillons les choix d'architecture, les schémas de conception, le code VHDL ainsi que les simulations associées à chaque module, afin de permettre une compréhension progressive du fonctionnement global du système et la manière dont moi je l'ai écrits.

- ✚ Unité Arithmétique et Logique (ALU);
- ✚ Banc de registre ;
- ✚ Datapath ;
- ✚ Multiplexeur 2 vers 1 ;
- ✚ Extension de Signe ;
- ✚ Assemblage de l'Unité de Traitement ;
- ✚ Unité de Gestion des Instruction ;
- ✚ Registre 32 bis avec commande de chargement ;
- ✚ Décodeur d'instructions ;
- ✚ Assemblage du processeur monocycle ;

La méthode de projet adoptée a consisté à organiser clairement les fichiers du projet selon une architecture proposée par Monsieur Yann DOUZE afin d'éviter toute confusion. Cette organisation s'est avérée plus efficace que celle que j'avais mise en place au premier semestre et a facilité le développement, la gestion des fichiers ainsi que les phases de simulation.

Projet_mono_multi_cycle

```
|—.git
|—.fit
|—.Docs
|—.simu
|—.src
|—.Rapport
```

Le dossier simu regroupe l'ensemble des bancs de test ainsi que les scripts de simulation permettant de valider le comportement des différents modules/composants VHDL. Le dossier fit contient les fichiers nécessaires à la synthèse et au déploiement du projet sur la carte DE10-Lite à l'aide du logiciel Quartus Prime. Le dossier src rassemble l'ensemble des fichiers VHDL

décrivant les différents composants du processeur, tandis que le dossier Rapport contient les documents associés à la rédaction du compte-rendu.

Par ailleurs, afin d'assurer un suivi du développement et de conserver un historique des modifications, un dépôt GitHub a été utilisé pour l'archivage du projet et ainsi pouvoir revenir en arrière en cas de problème.

Pour chacune de ces parties, il existe un fichier simu.do permettant d'automatiser les bancs de tests.

A. Unité Arithmétique et Logique (ALU)

L'Unité Arithmétique et Logique (ALU) développée est un circuit combinatoire 32 bits. Elle réalise des opérations arithmétiques ou des transferts directs de données en fonction d'un code opératoire, tout en mettant à jour des indicateurs d'état.

1. *Explication du code de l'ALU*

Le fonctionnement de l'ALU est contrôlé par un processus combinatoire sensible aux entrées A, B et OP dont le fichier est **alu.vhd**. Le signal OP sur 2 bits permet de sélectionner quatre comportements différents :

- ✚ "00" : addition ($S = A + B$). Les signaux sont convertis en unsigned pour effectuer l'opération arithmétique, puis réassignés sous forme de vecteurs de bits.
- ✚ "01" : passage de B ($S = B$). La sortie recopie directement l'opérande B.
- ✚ "10" : soustraction ($S = A - B$). L'opération est réalisée en logique non signée via le type unsigned.
- ✚ "11" : passage de A ($S = A$). La sortie recopie directement l'opérande A.

Entity: ALU

- File: alu.vhd

Diagram

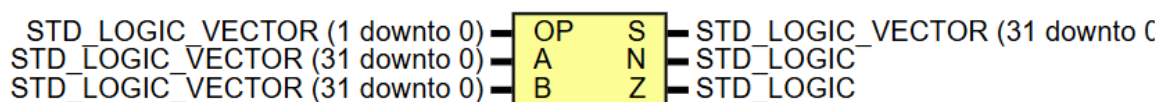


Figure 2 : Représentation graphique de l'ALU

Les drapeaux sont calculés en continu à partir du signal interne result_internal :

- ✚ Drapeau négatif (N) : il correspond au bit de poids fort du résultat (result_internal(31)). Si ce bit vaut '1', le résultat est négatif en complément à deux.
- ✚ Drapeau zéro (Z) : il passe à '1' si et seulement si les 32 bits du résultat sont tous à '0'.

2. Explication du banc de test de l'ALU

Le fichier **tb_alu.vhd** permet de vérifier le comportement de l'ALU sans entrées physiques, grâce à un scénario de simulation temporel.

Le composant ALU est instancié dans le testbench. Ses ports (OP, A, B, S, N et Z) sont connectés à des signaux locaux initialisés à zéro afin de garantir un état de départ connu.

Le processus de stimulation applique successivement plusieurs cas de test, séparés par un délai de 10 ns (wait for 10 ns) pour permettre la stabilisation des signaux.

Les tests réalisés sont les suivants : une addition ($10 + 5$), le transfert de l'opérande B, une soustraction ($10 - 5$), la vérification du flag Z avec l'opération $10 - 10 = 0$, la vérification du flag N avec l'opération $5 - 10 = -5$, puis le transfert de l'opérande A à l'aide de la valeur hexadécimale X"ABCDEFFF". Ces essais permettent de valider les principales fonctionnalités de l'ALU ainsi que le comportement des indicateurs d'état.

3. Résultat de la simulation de l'ALU

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme **simALU.do**.

La figure ci-dessous issue de ModelSim valide le comportement attendu de l'ALU à chaque étape de 10 ns. Les signaux A, B et S sont affichés en hexadécimal pour faciliter la lecture.

- ✚ 0 à 10 ns (addition) : OP = "00". A = 10 et B = 5. La sortie S vaut 15. Les flags N et Z restent à 0.
- ✚ 10 à 20 ns (passage de B) : OP = "01". S recopie B et vaut 5.
- ✚ 20 à 30 ns (soustraction) : OP = "10". Le calcul donne $10 - 5 = 5$.
- ✚ 30 à 40 ns (flag Z) : A = 10 et B = 10. Le résultat est 0, donc Z passe à 1.
- ✚ 40 à 50 ns (flag N) : A = 5 et B = 10. Le résultat est négatif (-5), donc N passe à 1 et Z revient à 0.
- ✚ 50 à 60 ns (passage de A) : OP = "11". A = X"ABCDEFFF". La sortie S recopie cette valeur. Le bit de poids fort étant à 1, le flag N reste actif.

Entrees ALU	Msgs								
/tb_alu/OP	-No Data-	2						3	
/tb_alu/A	-No Data-	0000000A				00000005		ABCDEFFF	
/tb_alu/B	-No Data-	00000005		0000000A					
Resultats									
/tb_alu/S	-No Data-	00000005		00000000		FFFFFFFB		ABCDEFFF	
Flags									
/tb_alu/N	-No Data-								
/tb_alu/Z	-No Data-								

Figure 3 : Simulation ALU

B. Banc de registre

1. Explication du code du banc de registre

À la mise sous tension ou lors d'un signal de reset, tous les registres de R0 à R14 sont mis à zéro. En revanche, le registre R15 (correspondant au compteur de programme ou PC) est initialisé avec la valeur 0x00000030 (48 en décimal), conformément aux directives du projet.

L'accès en lecture est purement combinatoire. Dès que les adresses RA ou RB changent, les données des registres correspondants sont immédiatement disponibles sur les bus de sortie A et B. Cela permet d'alimenter directement l'ALU sans attendre de front d'horloge.

L'écriture d'une donnée sur le bus W à l'adresse RW est séquentielle. Elle nécessite un front montant d'horloge (rising_edge(Clk)) ainsi que l'activation du signal d'autorisation d'écriture (WE = '1').

Entity: BANC_REGISTRE

• File: banc_registre.vhd

Diagram

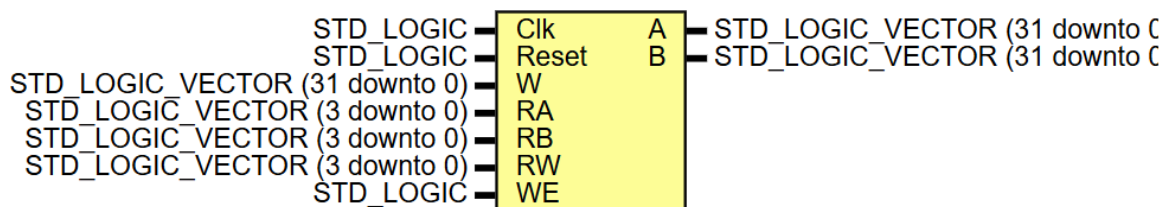


Figure 4 : Représentation graphique du Banc de registre

2. Explication du banc de test du banc de registre

Le banc de test fourni est **tb_datapath.vhd** (sans le V2), il instancie l'unité globale DataPath. Cependant, l'objectif de cette simulation est de valider uniquement le couplage entre le banc de registres et l'ALU. Par la même occasion montrée que le banc de registre fonctionne.

Remarque :

Bien que le fichier **datapath.vhd** contienne d'autres modules (comme la mémoire de données, l'extension de signe ou les multiplexeurs), ceux-ci ont été volontairement désactivés. L'intégration de ces éléments sera présentée plus tard dans le rapport.

Le scénario de test enchaîne cinq opérations arithmétiques consécutives, synchronisées par une horloge de période 10 ns :

- ✚ Copie du contenu de R15 (0x30) vers le registre R1 ;
- ✚ Première accumulation : somme $R1 + R15$ ($0x30 + 0x30$), résultat 0x60 stocké dans R1 ;
- ✚ Seconde accumulation : somme $R1 + R15$ ($0x60 + 0x30$), résultat 0x90 stocké dans R2 ;
- ✚ Première soustraction : $R1 - R15$ ($0x60 - 0x30$), résultat 0x30 stocké dans R3 ;
- ✚ Seconde soustraction (négative) : $R7 - R15$ ($0 - 0x30$), résultat -0x30, soit X"FFFFFFD0" en complément à deux.

3. *Résultat de la simulation du banc de registre*

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme **simDATAPATH.do (sans le V2)**.

La figure ci-dessous issue de ModelSim valide le comportement attendu du banc de registre et par la même occasion le datapath. Les valeurs des bus et des registres sont ici représentées au format hexadécimal.

- ✚ De 0 à 20 ns (Phase de Reset) : Le signal "rst" est actif à '1'. Le système s'initialise.
- ✚ De 20 à 30 ns (Stabilisation) : Le signal "rst" repasse à '0'. Le "BusB" affiche déjà 00000030 car l'adresse de lecture "RB" pointe sur X"F" (15) et la lecture est asynchrone.
- ✚ De 30 à 40 ns (Test 1 - Copie de "R15" dans "R1") : "RB" vaut F ("R15") et l'ALU est configurée en transfert de B ("aluctr" = 01). La sortie "Aluout" prend la valeur 00000030. Comme "MemtoReg" vaut 0, cette valeur est transmise sur "BusW". Au front montant d'horloge à 35 ns, elle est écrite dans "R1" ("RW" = 1).
- ✚ De 40 à 50 ns (Test 2 - " $R1 = R1 + R15$ ") : "RA" = 1 et "RB" = F. "BusA" et "BusB" affichent tous deux 00000030. L'ALU réalise une addition ("Aluotr" = 00) et produit 00000060 sur "Aluout" et "BusW". Cette valeur est enregistrée dans "R1" au front montant de 45 ns.
- ✚ De 50 à 60 ns (Test 3 - " $R2 = R1 + R15$ ") : "RA" reste à 1 (valeur 00000060) et "RB" = F (00000030). L'ALU calcule 00000090, présent sur "Aluout" et "BusW". Le résultat est écrit dans "R2" au front montant de 55 ns.

- De 60 à 70 ns (Test 4 - "R3" = "R1" - "R15") : L'ALU effectue une soustraction ("Alucontr" = 10) entre 00000060 et 00000030. Le résultat 00000030 est placé sur "Aluout" et écrit dans "R3" au front montant de 65 ns.
- De 70 à 80 ns (Test 5 - "R5" = "R7" - "R15") : "RA" = 7, donc "BusA" = 00000000, tandis que "BusB" = 00000030. L'ALU calcule 0 - 0x30, donnant FFFFFFFD0 en complément à deux. Cette valeur est transmise sur "BusW" et enregistrée dans "R5" au front montant de 75 ns. Le flag négatif "N_flag" passe à '1', confirmant le résultat négatif.

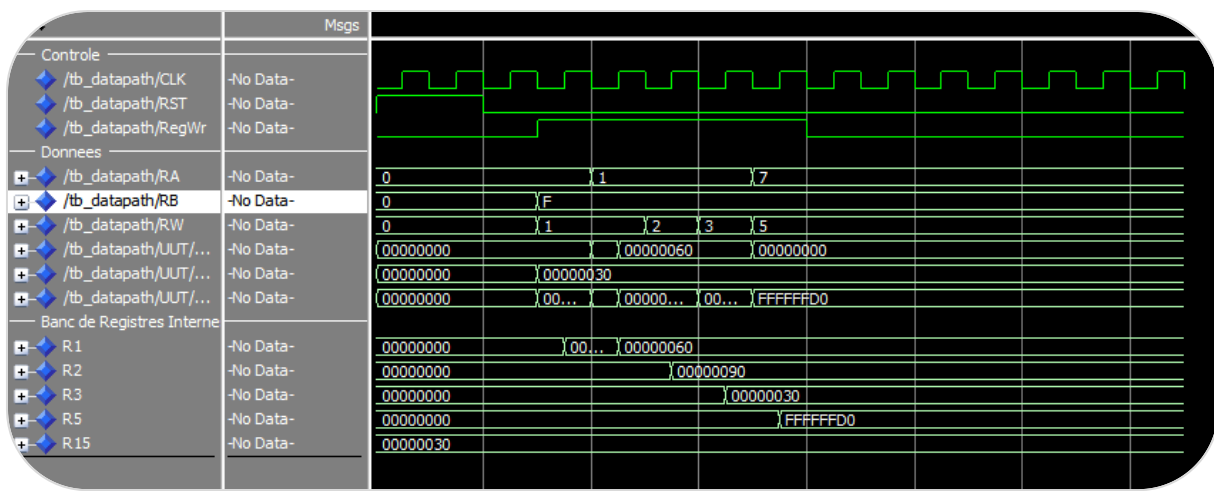


Figure 5 : Simulation Datapath (Banc de registre + ALU)

C. Le multiplexeur 2 vers 1

1. Explication du code du multiplexeur 2 vers 1

Le composant intègre un paramètre générique N_bits (fixé par défaut à 32). Cela permet de le réutiliser à différents endroits du Datapath avec des tailles de bus différentes, sans avoir à réécrire le code.

L'architecture utilise une affectation conditionnelle directe (when... else). Le signal de commande COM détermine quelle entrée est envoyée vers la sortie S :

- Si COM = '0', la sortie S recopie le bus d'entrée A ;
- Si COM = '1', la sortie S recopie le bus d'entrée B ;

Entity: MUL_2_TO_1

- File: mul_2_to_1.vhd

Diagram

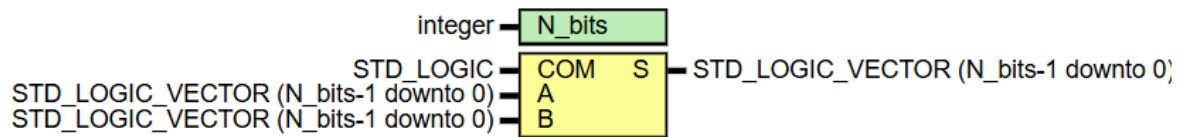


Figure 6 : Représentation graphique du Multiplexeur 2 vers 1

2. Explication du banc de test du multiplexeur 2 vers 1

Le fichier **tb_mul_2_to_1.vhd** permet de vérifier le comportement de du multiplexeur 2 vers 1.

Le banc de test instancie le multiplexeur en fixant la valeur de N_bits à 32 afin de simuler l'aiguillage de mots de données complets.

Le processus applique une série de stimuli temporels espacés de 10 ns pour valider les différentes configurations du multiplexeur :

- ✚ Initialisation (COM = 0) : application de deux motifs de bits alternés (A = 0xAAAAAAAA et B = 0x55555555) ;
- ✚ Bascule (COM = 1) : maintien des mêmes entrées afin d'observer le changement de la sortie ;
- ✚ Nouvelles valeurs (COM = 0) : modification simultanée des entrées A et B avec de nouvelles valeurs (0x12345678 et 0x87654321) ;
- ✚ Dernière bascule (COM = 1) : changement final de la commande afin de vérifier la transmission correcte de la valeur de B.

3. Résultat de la simulation du multiplexeur 2 vers 1

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme **simuMUL_2_TO_1.do**.

La figure ci-dessous issue de ModelSim valide le comportement attendu du multiplexeur. Les valeurs des bus A, B et S sont affichées en notation hexadécimale et confirment le fonctionnement combinatoire et instantané du composant.

- ✚ De 0 à 10 ns (test 1) : la commande com est positionnée à '0'. L'entrée A vaut AAAAAAAAAA et B vaut 55555555. Fidèle à sa table de vérité, le multiplexeur sélectionne l'entrée A : la sortie S prend immédiatement la valeur AAAAAAAAAA.
- ✚ De 10 à 20 ns (test 2) : les entrées A et B restent inchangées, mais le signal de commande com passe à '1'. Le multiplexeur bascule instantanément : la sortie S ne copie plus A mais B, prenant la valeur 55555555.
- ✚ De 20 à 30 ns (test 3) : le signal de commande com repasse à '0' pendant que les entrées reçoivent de nouvelles valeurs : A prend 12345678 et B prend 87654321. La commande étant à '0', c'est le nouveau contenu de A qui est sélectionné : la sortie S affiche 12345678.
- ✚ De 30 à 40 ns (test 4) : enfin, la commande com repasse à '1'. Le circuit combinatoire réagit sans aucun retard d'horloge et propage la nouvelle valeur de B : la sortie S se stabilise à 87654321.

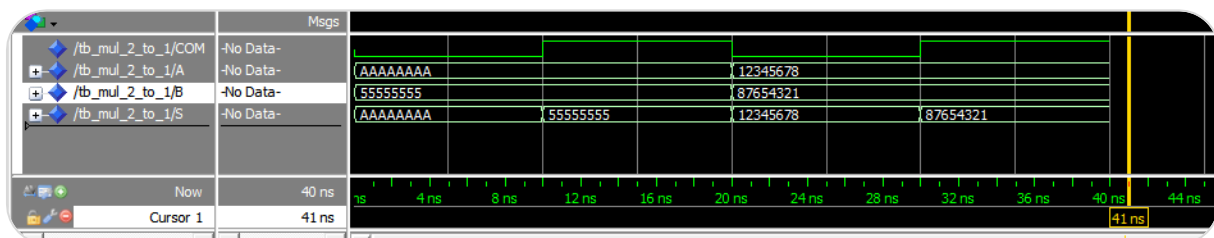


Figure 7 : Simulation du Multiplexeur 2 vers 1

D. Extension de signe

1. Explication du code de l'extension du signe

Le module d'extension de signe (SIGN_EXTENSION) est un circuit combinatoire utilisé pour transformer une valeur immédiate de taille réduite en un format 32 bits compatible avec le fonctionnement interne de l'ALU.

La généricité du module, via le paramètre « N_bits_E », permet d'adapter facilement sa taille d'entrée. Dans notre cas, l'entrée est configurée sur 8 bits, mais le module peut également être utilisé pour d'autres architectures nécessitant des largeurs différentes, par exemple 12 ou 24 bits dans certains formats d'instructions de type ARM.

Le principe de l'extension de signe repose sur la conservation de la valeur en complément à deux lors du passage à une taille supérieure. Pour cela, le bit de poids fort de l'entrée (MSB), qui représente le signe, est recopié sur l'ensemble des bits ajoutés. Cela garantit que la valeur reste inchangée, qu'elle soit positive ou négative.

L'architecture repose sur un processus combinatoire sensible à "E". Les bits de poids faible de la sortie "S" (de 0 à N_bits_E - 1) recopient directement les bits de l'entrée "E". Les bits restants (de N_bits_E à 31) sont remplis avec la valeur du bit de signe de "E" grâce à l'expression (others => E(N_bits_E - 1)).

Entity: SIGN_EXTENSION

- File: sign_extension.vhd

Diagram

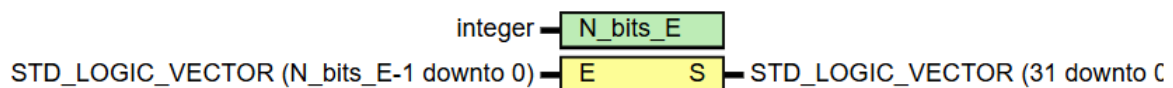


Figure 8 : Représentation graphique de l'Extension de signe

2. Explication du banc de test du multiplexeur 2 vers 1

Le fichier **tb_sign_extension.vhd** permet de vérifier le comportement du composant.

Le banc de test configure le paramètre « N_bits_E » à 8 afin de simuler l'extension d'un octet signé vers un mot de 32 bits.

Le processus applique quatre profils de bits différents toutes les 10 ns :

- ✚ Valeur positive (00000101) : équivaut à +5 en décimal. Le bit de signe (MSB) vaut '0' ;
- ✚ Valeur négative (11111011) : équivaut à -5 en décimal. Le bit de signe (MSB) vaut '1' ;
- ✚ Autre valeur négative (10000001) : équivaut à -127 en décimal. Le bit de signe (MSB) vaut '1' ;
- ✚ Zéro (00000000) : cas limite, le bit de signe vaut '0'.

3. Résultat de la simulation du multiplexeur 2 vers 1

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme ***simu*** ***simuSIGN_EXTENSION.do***.

La figure ci-dessous issue de ModelSim montre le bon fonctionnement de l'extension de signe. L'entrée *e* est affichée en binaire afin d'observer son bit de signe, tandis que la sortie *s* sur 32 bits est affichée en notation binaire.

- ✚ De 0 à 10 ns (test 1 - entrée positive) : l'entrée *e* reçoit "00000101". Le bit de signe vaut '0'. Le module propage donc des '0' sur les bits supérieurs de la sortie. On observe sur le bus *s* la valeur 00000005. La valeur +5 est conservée.
- ✚ De 10 à 20 ns (test 2 - entrée négative) : l'entrée *e* passe à "11111011". Le bit de signe vaut '1'. Le module l'identifie et remplit les bits supérieurs de la sortie avec des '1'. En hexadécimal, ces bits forment la valeur FFFFFFFB, ce qui correspond à -5 en complément à deux sur 32 bits.
- ✚ De 20 à 30 ns (test 3 - seconde entrée négative) : l'entrée *e* prend la valeur "10000001". Le bit de poids fort étant toujours à '1', l'extension applique la même règle de propagation des '1'. La sortie *s* devient FFFFFFF8, en conservant la valeur négative.
- ✚ De 30 à 40 ns (test 4 - zéro) : l'entrée *e* vaut "00000000". Le bit de signe est à '0', donc la sortie est complétée avec des zéros. La sortie *s* affiche 00000000.

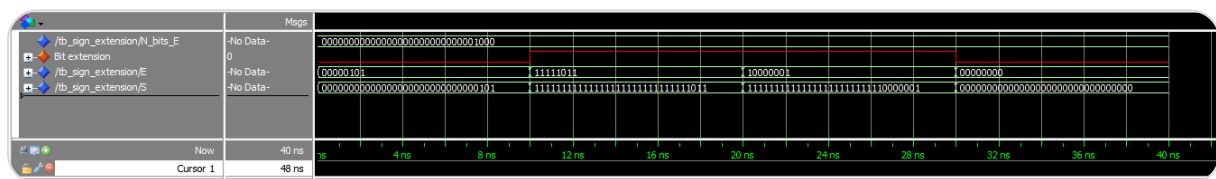


Figure 9 : Simulation de l'Extension du signe

E. Mémoire de données

1. Explication du code sur la mémoire de données

Le module DATA_MEMORY modélise une mémoire vive (RAM) de type synchrone en écriture et asynchrone en lecture, structurée pour stocker 64 mots de 32 bits.

La mémoire est définie sous la forme d'un tableau de 64 éléments (0 to 63), chacun codé sur 32 bits.

À l'état initial ou lors de l'activation du signal reset, la mémoire charge un profil de données spécifique sur les adresses allant de 16 (0x10) à 26 (0x1A), tandis que le reste des cases est initialisé à zéro. Ce pré-chargement simule la présence de valeurs initiales nécessaires à l'exécution des programmes (comme des données de test ou des zones de stockage pour l'instruction STR).

L'écriture de la donnée DataIn à l'adresse Addr nécessite la validation du signal d'écriture (WrEn = '1') et s'effectue au front montant de l'horloge (rising_edge(CLK)).

Contrairement à l'écriture, la sortie DataOut est directement connectée à la case mémoire pointée par Addr. Tout changement d'adresse se traduit immédiatement par la mise à jour de la donnée en sortie, ce qui permet de réduire le temps d'accès lors des cycles de lecture (instructions de type LDR).

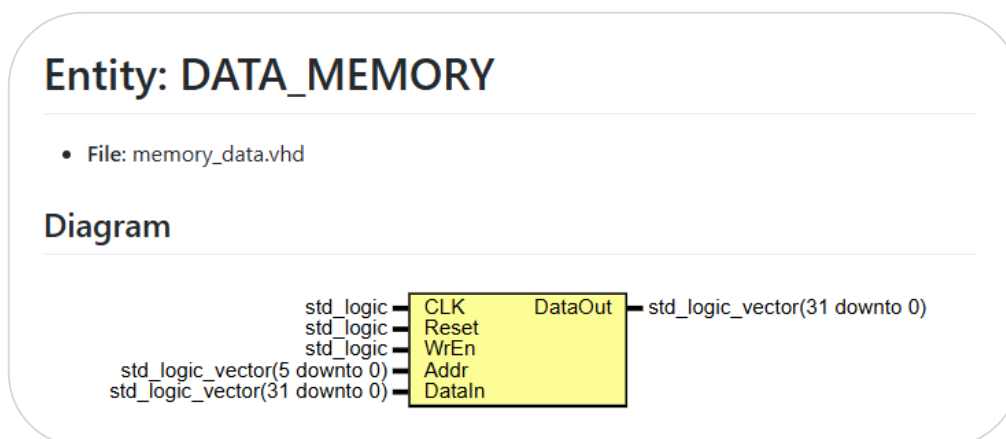


Figure 10 : Représentation graphique de la Memory data

2. Explication du banc de test sur la mémoire de données

Le fichier **tb_memory_data.vhd** permet de vérifier le comportement du composant.

Le banc de test génère une horloge de période 10 ns

- ✚ Phase de reset : activation du signal Reset pendant un cycle complet afin d'initialiser le tableau avec les valeurs par défaut ;
- ✚ Ecriture à l'adresse 0 : configuration de l'adresse 000000, injection du motif de test 0xAAAAAAAA et activation du signal WrEn ;
- ✚ Maintien et lecture : désactivation de WrEn afin de vérifier que la donnée reste stable en mémoire ;
- ✚ Ecriture à l'adresse 10 (0x0A) : configuration de l'adresse 001010, injection de la valeur 0x12345678 et activation de WrEn.

3. *Résultat de la simulation de la mémoire de donnée*

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme ***simuMEMORY_DATA.do***.

La figure ci-dessous issue de ModelSim montre le bon fonctionnement de la mémoire de donnée.

- ✚ De 0 à 10 ns (phase de reset) : le signal reset est actif à '1'. Bien que l'adresse addr pointe sur 00, la sortie dataout affiche 00000000 car la case mémoire 0 vient d'être initialisée à zéro par le bloc de reset.
- ✚ De 10 à 20 ns (écriture à l'adresse 0) : le reset retombe à '0'. L'adresse addr reste à 00, datain présente la valeur AAAAAAAAA et wren passe à '1'. Au front montant de l'horloge situé à 15 ns, l'écriture est validée en interne. Par conséquent, la lecture asynchrone recopie immédiatement cette modification : la sortie dataout bascule à AAAAAAAAA.
- ✚ De 20 à 30 ns (maintien de l'adresse 0) : le signal wren est remis à '0'. La sortie dataout maintient sa valeur AAAAAAAAA, ce qui montre que la donnée a bien été stockée dans la case 0 de la mémoire et qu'aucune écriture supplémentaire n'a lieu.
- ✚ De 30 à 40 ns (écriture à l'adresse 10 / 0x0A) : l'adresse addr bascule à 0A (10 en décimal). Avant même le front d'horloge, on observe que dataout passe instantanément à 00000000 (lecture asynchrone de la case 10 initialement vide). Le signal datain présente 12345678 et wren remonte à '1'. Au front montant de l'horloge à 35 ns, la donnée est enregistrée en mémoire. La sortie dataout affiche alors instantanément le nouveau contenu de la case : 12345678.
- ✚ Après 40 ns (fin de simulation) : le signal wren retombe à '0', ce qui stabilise le système et valide la réussite de l'ensemble des opérations d'accès à la mémoire.

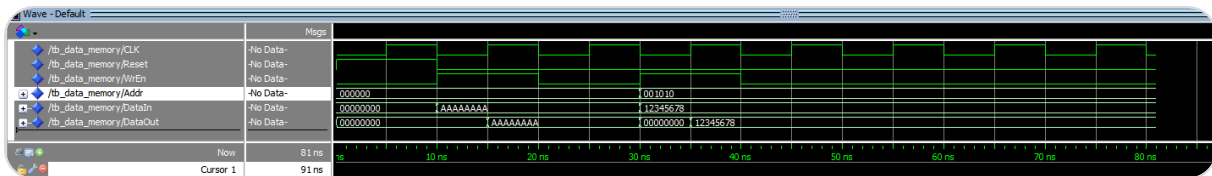


Figure 11 : Simulation de la Memory Data

F. Assemblage Unité de Traitement

Après avoir validé individuellement chaque bloc périphérique (multiplexeurs, extension de signe, mémoire de données) et l'interaction de base entre le banc de registres et l'ALU (avec la première version du datapath qui devient la version finale du datapath avec tous les blocs), cette étape marque la complétion d'une partie du processeur monocycle.

Entity: DataPath

- File: datapath.vhd

Diagram



Figure 12 : Représentation graphique du Datapath

Voici l'assemblage effectué avec les différents composants fait précédemment.

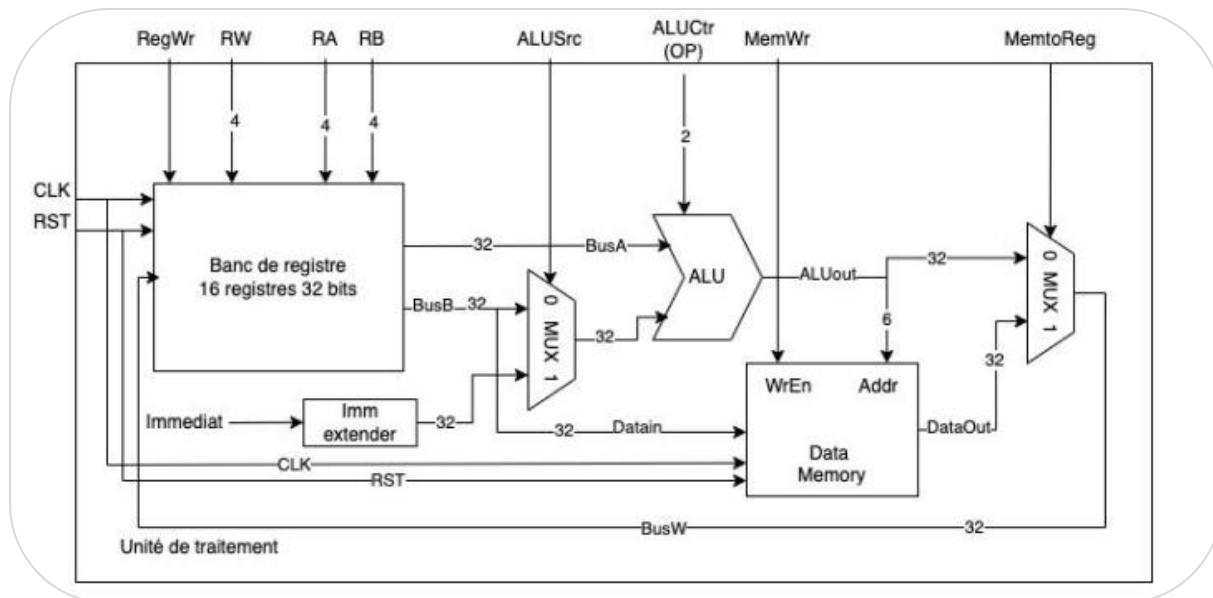


Figure 13 : Assemblage des modules créer précédement

Remarque :

Vous verrez dans le fichier **datapath.vhd**, qu'il y a « RegAff_en » et « RegAff_out » qui sont apparent mais non utiliser, c'est normal car à ce stade du projet il n'était pas demandé de la faire.

1. Explication du banc de test du datapath

Le fichier **tb_datapathV2.vhd** permet de vérifier le comportement du datapath.

Les différents tests réalisés sont les suivants :

- ✚ Phase d'initialisation : chargement des valeurs de départ avec R1 = 10 (0x0A) et R2 = 20 (0x14).
- ✚ Test 1 : addition registre-registre : calcul de $R3 = R1 + R2$, soit $10 + 20 = 30$ (0x1E).
- ✚ Test 2 : addition registre-immédiat : calcul de $R4 = R1 + 5$, soit 15 (0x0F).
- ✚ Test 3 : soustraction registre-registre : calcul de $R5 = R2 - R1$, soit $20 - 10 = 10$ (0x0A).
- ✚ Test 4 : soustraction registre-immédiat : calcul de $R6 = R2 - 3$, soit 17 (0x11).
- ✚ Test 5 : copie d'un registre : transfert de la valeur de R1 dans R7 en réalisant l'opération $R1 + 0$.
- ✚ Test 6 : écriture en mémoire (STR) : stockage de la valeur de R2 (20) à l'adresse contenue dans R1 (10).
- ✚ Test 7 : lecture mémoire (LDR) : récupération de la valeur stockée à l'adresse 10 et écriture du résultat dans R8.

2. Résultat de la simulation du datapath

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme **simuDATAPATHV2.do**.

La figure ci-dessous issue de ModelSim montre le bon fonctionnement du datapath.

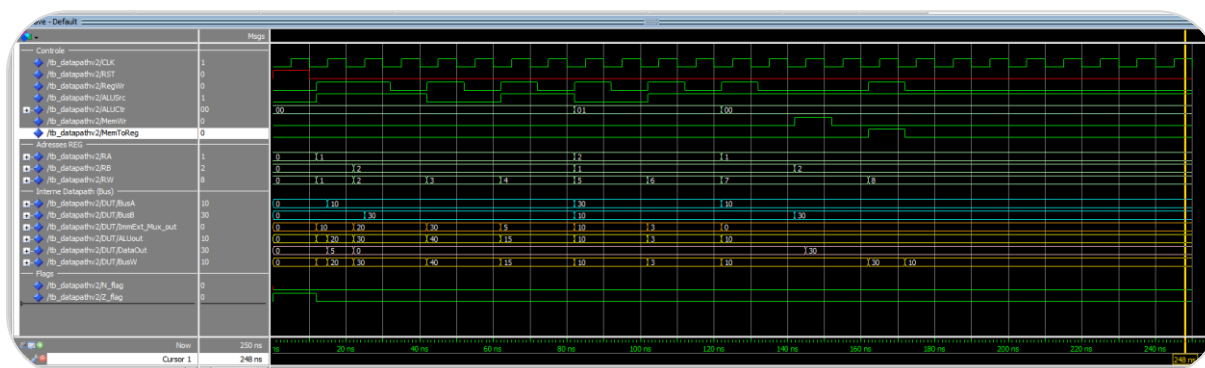


Figure 14 : Simulation du Datapaht

G. Registre de commande 32bits

Pour des raisons de simplification du code, j'ai décidé d'intégrer le registre de commande sur 32 bits directement au sein du **datapath.vhd**. En effet, je ne souhaitais pas créer de fils (signals) supplémentaires ni ajouter une nouvelle instanciation de composant dans mon architecture.

Bien que ce choix puisse potentiellement avoir des répercussions ou des impacts structurels ailleurs dans le processeur, je trouve que le faire de cette manière rend le système beaucoup plus clair. Cela m'offre une bien meilleure lisibilité globale du code et me permet de mieux visualiser le comportement et le flux des données lors des phases de routage et de simulation.

Le fichier **tb_ref_aff.vhd** permet de vérifier le comportement dans le datapath.

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme **simuREG_AFF.do**.

On observe d'après la figure ci-dessous extraite de ModelSim, le bon fonctionnement de notre registre de commande de 32 bits.

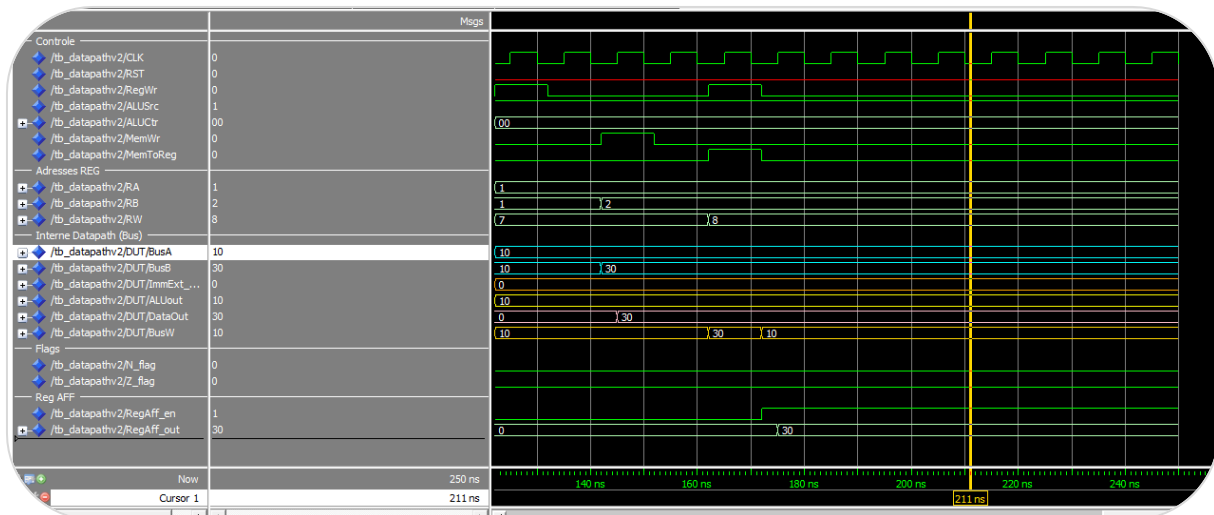


Figure 15 : Simulation du registre de commande 32 bits dans le datapath

H. Unité de gestion d'instruction

1. Explication du code de l'Unité de Gestion d'Instruction

L'unité de gestion des instructions a pour rôle de générer l'adresse de l'instruction à exécuter, de récupérer cette instruction depuis la mémoire d'instructions et de calculer l'adresse de la prochaine instruction.

Le registre PC (« PC_reg ») est un registre synchrone de 32 bits mis à jour à chaque front montant d'horloge. Lorsque le signal Reset est activé, sa valeur est réinitialisée à l'adresse 0x00000000.

Le module d'extension de signe permet de convertir l'offset de branchement codé sur 24 bits en une valeur signée sur 32 bits. Cela permet de réaliser des sauts aussi bien vers l'avant que vers l'arrière dans le programme.

Le multiplexeur de sélection de la prochaine adresse (« nPCsel ») détermine l'adresse de la prochaine instruction à exécuter :

- ✚ si « nPCsel » = '0', le processeur fonctionne de manière séquentielle et le compteur de programme s'incrémente de 1 à chaque cycle ;
- ✚ si « nPCsel » = '1', le processeur exécute une instruction de branchement. Dans ce cas, l'adresse suivante est calculée à partir de l'offset contenu dans l'instruction. Pour les offsets négatifs, une logique spécifique permet de revenir vers certaines parties du programme, notamment pour réaliser des boucles. Pour les offsets positifs, l'adresse suivante est calculée en ajoutant l'offset étendu à la valeur actuelle du compteur de programme.

Entity: UNIT_GESTION_INSTRUCTION

• File: unit_gestion_instruction.vhd

Diagram

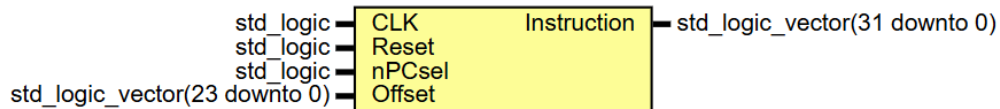


Figure 16 : : Représentation graphique de l'Unité de Gestion d'Instruction

2. Explication du banc de test de l'Unité de Gestion d'Instruction

Le fichier **tb_unit_gestion_intruction.vhd** permet de vérifier le comportement du composant.

Le banc de test applique une horloge de 10 ns pour valider le comportement du pointeur d'instruction à travers quatre scénarios successifs :

- ✚ Initialisation (0 à 10 ns) : Forçage du Reset pour caler le PC à 0.
- ✚ Mode Linéaire (10 à 40 ns) : Désactivation du saut (nPCsel = '0') pour observer le PC s'incrémenter naturellement (0, 1, 2, 3).
- ✚ Saut en Avant (40 à 50 ns) : Forçage de « nPCsel » = '1' avec un offset positif de 0x000002.
- ✚ Saut en Arrière (60 à 70 ns) : Application d'un offset négatif de 0xFFFFF (équivalent à -1 en complément à deux) avec « nPCsel » = '1'.

3. Résultat de la simulation de l'Unité de Gestion d'Instruction

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme **simUGI.do**.

La figure ci-dessous issue de ModelSim valide le comportement attendu de l'unité de gestion des instructions. Les signaux pc_reg, pc_suivant et instruction permettent de suivre l'évolution du compteur de programme ainsi que les instructions lues en mémoire.

✚ De 0 à 15 ns :

Le signal reset est actif à '1' jusqu'à 10 ns. Au premier front d'horloge à 5 ns, le registre pc_reg est réinitialisé à 00000000. La mémoire d'instructions fournit alors

immédiatement l'instruction située à cette adresse, soit E3A01010. Dans le même temps, le signal pc_suivant prépare déjà l'adresse suivante, soit 00000001.

✚ De 15 à 45 ns :

Le signal nPCsel reste à '0', ce qui permet au compteur de programme de s'incrémenter normalement à chaque cycle.

- à 15 ns, pc_reg passe à 00000001 et l'instruction lue devient E3A02014 ;
- à 25 ns, pc_reg passe à 00000002 et l'instruction lue est E5913000 ;
- à 35 ns, pc_reg passe à 00000003 et l'instruction lue est E0834002.
- À chaque étape, le signal pc_suivant prépare la valeur PC + 1.

✚ De 45 à 55 ns (test du saut en avant) :

À 40 ns, le banc de test applique un offset de 000002 et positionne nPCsel à '1'. La logique de branchement calcule alors une nouvelle adresse à partir du PC courant et de l'offset. Comme pc_reg vaut 3 et que l'offset vaut 2, le signal pc_suivant prend la valeur 00000005.

Au front montant de 45 ns, cette nouvelle adresse est chargée dans pc_reg. Le compteur passe directement à 00000005 et l'instruction lue devient E2811001.

✚ De 55 à 65 ns (retour au mode linéaire) :

Le signal nPCsel repasse à '0' à 50 ns. Le compteur reprend alors son fonctionnement normal. Au front d'horloge de 55 ns, pc_reg passe à 00000006 et l'instruction lue est EA000002.

✚ De 65 à 75 ns (test du saut en arrière) :

À 60 ns, nPCsel repasse à '1' avec un offset de FFFFFFFF, correspondant à -1 en complément à deux. Après extension de signe, la valeur obtenue est FFFFFFFF.

La logique de branchement détecte alors un offset négatif et applique la règle de retour définie dans le projet. Le signal pc_suivant prend immédiatement la valeur 00000000.

Au front d'horloge de 65 ns, le compteur revient au début du programme. Le registre pc_reg affiche à nouveau 00000000 et la première instruction, E3A01010, est de nouveau lue en mémoire.

Le PC s'incrémente donc de 1 à chaque front montant d'horloge, exactement comme au début du programme, de 75 ns à 180ns.



Figure 17 : Résultat de la simulation de l'Unité de Gestion d'Instruction

I. Décodeur

1. Explication du code du décodeur

Le module DECODEUR constitue l'Unité de Contrôle (Control Unit) du processeur monocycle. Il s'agit d'un circuit purement combinatoire. Son rôle est de traduire l'instruction binaire de 32 bits actuellement lue en mémoire, ainsi que les drapeaux d'état (Flags_NZCV), en un ensemble de signaux de contrôle. Ces signaux pilotent le chemin de données (Datapath) ainsi que l'unité de gestion des instructions.

Découpage du mot d'instruction (décodage amont)

Avant d'entrer dans la logique de décision, le décodeur extrait les champs principaux de l'instruction ARM simplifiée :

- ✚ cond = instruction(31 downto 28) : code de condition de l'instruction (par exemple 1110 pour Always ou 1011 pour Less Than).
- ✚ bit_I = instruction(25) : indique si le second opérande est une valeur immédiate (1) ou un registre (0).
- ✚ opcode = instruction(24 downto 21) : code de l'opération arithmétique ou logique (par exemple 1101 pour MOV ou 0100 pour ADD).
- ✚ flag_N = Flags_NZCV(3) : extraction du drapeau Négatif, utilisé notamment pour le traitement du saut conditionnel BLT.

Entity: DECODEUR

- File: decodeur.vhd

Diagram

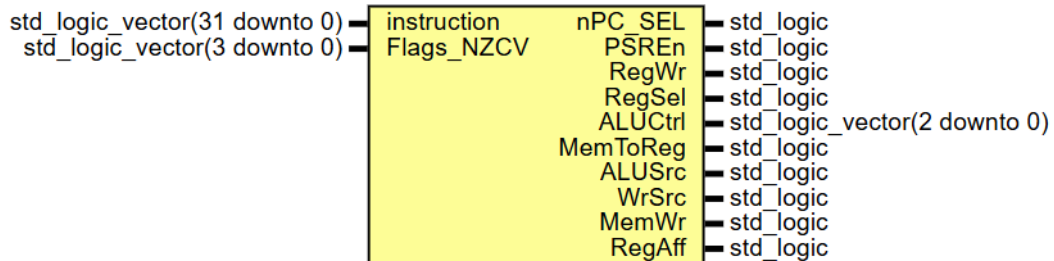


Figure 18 : Représentation graphique du Décodeur

a) Valeur des commandes

Voici les valeurs de commandes que j'ai trouvé, sachant que le « - » signifie peut importe

Instruction	nPCSel	RegWr	ALUSrc	ALUCtr	PSREn	MemWr	WrSrc
ADDi	0	1	1	Add	0	0	0
ADDr	0	1	0	Add	0	0	0
BAL	1	0	-	-	0	0	-
BLT	N	0	-	-	0	0	-
CMP	0	0	1	sub	1	0	-
LDR	0	1	1	Add	0	0	1
MOV	0	1	1	mov	0	0	0
STR	0	0	1	Add	0	1	-

Figure 19 : Valeur des commandes

2. Explication du banc de test de l'Unité de Gestion d'Instruction

Le fichier **tb_decodeur.vhd** permet de vérifier le comportement du composant.

Le scénario de test a été construit pour balayer l'ensemble du jeu d'instructions :

- ✚ **Les opérations arithmétiques et logiques** : Validation du mode immédiat (MOV, CMP) et du mode registre (ADD).
- ✚ **Les accès mémoire** : Vérification du comportement asymétrique entre la lecture (LDR) et l'écriture (STR).
- ✚ **Les branchements** : Test du saut incondtionnel (BAL) et du saut conditionnel (BLT). Pour ce dernier, le banc de test applique une double vérification cruciale : il injecte la même instruction à deux instants différents en modifiant uniquement le vecteur de drapeaux

(Flags_NZCV = "0000" puis "1000"), afin de valider que la décision de saut ne s'active *que* si le drapeau négatif (N) est levé.

Chaque instruction est maintenue stable pendant 20 ns, un laps de temps largement suffisant permettant une lecture avec une certaine aisance sur le chronogramme. La simulation prend fin grâce à l'instruction wait; finale qui gèle indéfiniment le processus.

3. Résultat de la simulation de l'Unité de Gestion d'Instruction

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme **simDECODEUR.do**.

La figure ci-dessous issue de ModelSim montre le bon fonctionnement du décodeur.

- ✚ 10 ns à 30 ns – Instruction MOV R1, #0x10 (Opcode : E3A01010) : Le décodeur reconnaît une instruction MOV. L'écriture dans le banc de registres est activée (« RegWr » = '1') et l'opérande immédiat est sélectionné (« ALUSrc » = '1'). L'ALU est configurée en mode transfert (« ALUCtrl » = "001"). Le chronogramme confirme la stabilité de ces signaux sur toute la durée de l'opération.
- ✚ 30 ns à 50 ns – Instruction MOV R2, #0 (Opcode : E3A02000) : Cette instruction produit le même comportement que la précédente. Les signaux « RegWr », « ALUSrc » et « ALUCtrl » conservent respectivement les valeurs '1', '1' et "001", ce qui est cohérent avec une nouvelle opération MOV.
- ✚ 50 ns à 70 ns – Instruction LDR R0, 0(R1) (Opcode : E4110000) : L'instruction réalise un chargement mémoire. L'adresse est calculée par addition (« ALUSrc » = '1', « ALUCtrl » = "000"), tandis que « MemToReg » = '1' redirige les données lues vers le banc de registres. Le signal « RegWr » reste actif afin de permettre l'écriture du résultat.
- ✚ 70 ns à 90 ns – Instruction ADD R2, R2, R0 (Opcode : E0822000) : L'ALU effectue une addition entre deux registres (« ALUCtrl » = "000"). Le second opérande provient du banc de registres, ce qui entraîne le passage de « ALUSrc » à '0'. L'écriture du résultat est autorisée par « RegWr » = '1'.
- ✚ 90 ns à 110 ns – Instruction CMP R1, #0x1A (Opcode : E351001A) : Cette comparaison met à jour les drapeaux sans modifier de registre. Le signal « RegWr » passe à '0', tandis que « PSREn » = '1' autorise la mise à jour du registre d'état. L'ALU réalise une soustraction (« ALUCtrl » = "010") avec une valeur immédiate.

- ✚ 110 ns à 130 ns – Instruction BLT loop (Opcode : BAFFFFFFB), avec N = '0' : La condition de branchement n'est pas satisfaite puisque le drapeau négatif est à '0'. Le signal « nPC_SEL » reste donc à '0', ce qui maintient l'exécution séquentielle du programme.
- ✚ 130 ns à 150 ns – Instruction BLT loop (Opcode : BAFFFFFFB), avec N = '1' : Le drapeau négatif passe à '1', validant la condition de branchement. Le signal « nPC_SEL » est alors activé ('1'), indiquant que le processeur charge l'adresse cible du saut.
- ✚ 150 ns à 170 ns – Instruction STR R2, 0(R1) (Opcode : E4012000) : Cette instruction écrit le contenu de R2 en mémoire. Le signal « MemWr » = '1' autorise l'écriture mémoire tandis que « RegWr » = '0' confirme qu'aucun registre n'est modifié. Le signal « RegSel » = '1' sélectionne R2 comme source des données à stocker.



Figure 20 : Résultat de la simulation du Decodeur

III. Validation du processeur monocycle

Cette étape finale concrétise le projet en interconnectant les trois entités principales au sein du module de plus haut niveau **proc_mono_cycle.vhd** : l'Unité de Gestion des Instructions, l'Unité de Traitement (DataPath), ainsi que l'Unité de Contrôle (DECODEUR).

L'objectif de cette intégration est d'assurer la communication entre les différents blocs du processeur afin de permettre l'exécution des instructions au sein de cet architecture monocycle.

A. Explication du banc de test

Pour valider le comportement du processeur complet, un banc de test global a été mis en œuvre. Contrairement aux entités précédemment développées, ce testbench présente une structure volontairement simple.

Le banc de test et le script de simulation se nomme : **tb_proc_mono_cycle.vhd** et **simuPROC_MONO.do**.

- ✚ Génération d'horloge : le processus « clk_process » génère un signal d'horloge périodique stabilisé à 50 MHz (période « CLK_PERIOD » = 20 ns).
- ✚ Séquence de réinitialisation : le processus « stim_proc » applique une impulsion de Reset au démarrage afin de forcer le compteur de programme (PC) et le registre d'état (PSR) à des valeurs initiales connues, avant de laisser le processeur s'exécuter en totale autonomie (wait;).

L'autonomie de ce banc de test est totale, car le processeur ne dépend d'aucun stimulus externe pendant son fonctionnement.

B. Programme exécuté par le processeur monocycle

```

0x0 main : MOV R1,#0x10      ; --R1 <= 0x10
0x1      MOV R2,#0          ; --R2 <= 0
0x2 loop : LDR R0,0(R1)      ; --R0 <= DATA_MEM[R1]
0x3      ADD R2,R2,R0        ; --R2 <= R2 + R0
0x4      ADD R1,R1,#1        ; --R1 <= R1 + 1
0x5      CMP R1,0x1A         ; -- R1 - 0x1A, mise à jour de N
0x6      BLT loop           ; --branchement à _loop si R1 inferieur a 0x1A
0x7 end   : STR R2,0(R1)     ; --DATA_MEM[R1] <= R2
0x8      BAL main           ; --branchement à _main

```

Figure 21 : Programme exécuté par le processeur monocycle

Pour que ce programme s'exécute de manière assez bien, la mémoire de données (Data_Memory) a été pré-remplie avec une suite de valeurs incrémentales à partir de l'adresse 0x10 jusqu'à l'adresse de fin 0x1A (comme demandée dans le sujet):

- ✚ Données injectées : 0x000000FF à l'adresse 0x10, puis 2, 3, 4, 5, 6, 7, 8, 9, 10 pour les adresses suivantes entre les adresses 0x11 à 0x19.
- ✚ Adresse de sauvegarde (0x1A) : Initialisée à 0x0000000B, elle servira de réceptacle pour l'instruction finale STR.

C. Analyse de la simulation

Le chronogramme obtenu sous ModelSim illustre le comportement à long terme du processeur sur une fenêtre de 3000 ns. On y observe le régime cyclique induit par l'instruction de branchement conditionnel BLT.

Dans la section « Suivi des Registres de Travail », on identifie clairement l'évolution de nos variables:

- ✚ Le registre R1 (Pointeur) commence sa course à 0x00000010 et s'incrémente d'une unité à chaque passage dans la boucle (instruction à l'adresse 0x4).
- ✚ Le registre R0 charge successivement les valeurs présentes en mémoire (0xFF, puis 2, 3, etc.).
- ✚ Le registre R2 (Accumulateur) totalise les valeurs lues. Après le premier cycle, il accueille la valeur 0xFF (255), puis croît successivement au rythme des additions successives.

Concernant le signal Flag N (Négatif), lors de l'exécution de l'instruction CMP R1, 0x1A (adresse 0x5), tant que le pointeur R1 est inférieur à 0x1A, la soustraction interne $R1 - 0x1A$ donne un résultat négatif. Le drapeau Flag N se lève donc à 1 à la fin de chaque itération.

Ce maintien du drapeau à 1 force l'Unité de Contrôle à lever le signal nPC_SEL au cycle suivant, provoquant le saut du PC vers l'adresse de la boucle 0x2 au lieu de continuer linéairement.

Dès que le registre R1 atteint la valeur limite 0x1A (visible aux alentours de 1000 ns sur le chronogramme), le résultat de la comparaison devient nul. Le drapeau Flag N tombe à 0, rompant ainsi la condition du BLT.

Le processeur bascule sur l'instruction de fin STR R2, 0(R1) à l'adresse 0x7 : Le signal MemWr passe brièvement à 1, validant l'écriture du résultat final accumulé dans le registre R2 vers la case mémoire 0x1A.

Simultanément, le signal RegAff s'active, capturant la valeur finale calculée pour l'injecter sur le bus interne d'affichage s_RegAff_val.

En bas du chronogramme, les lignes de l'afficheur HEX (HEX0 à HEX3) traduisent immédiatement cette valeur finale stable en combinatoire pour piloter les afficheurs 7 segments de la carte FPGA.

Le programme termine sa course sur l'instruction BAL main (0x8) qui réinitialise proprement le compteur de programme vers le label main, relançant indéfiniment l'algorithme.

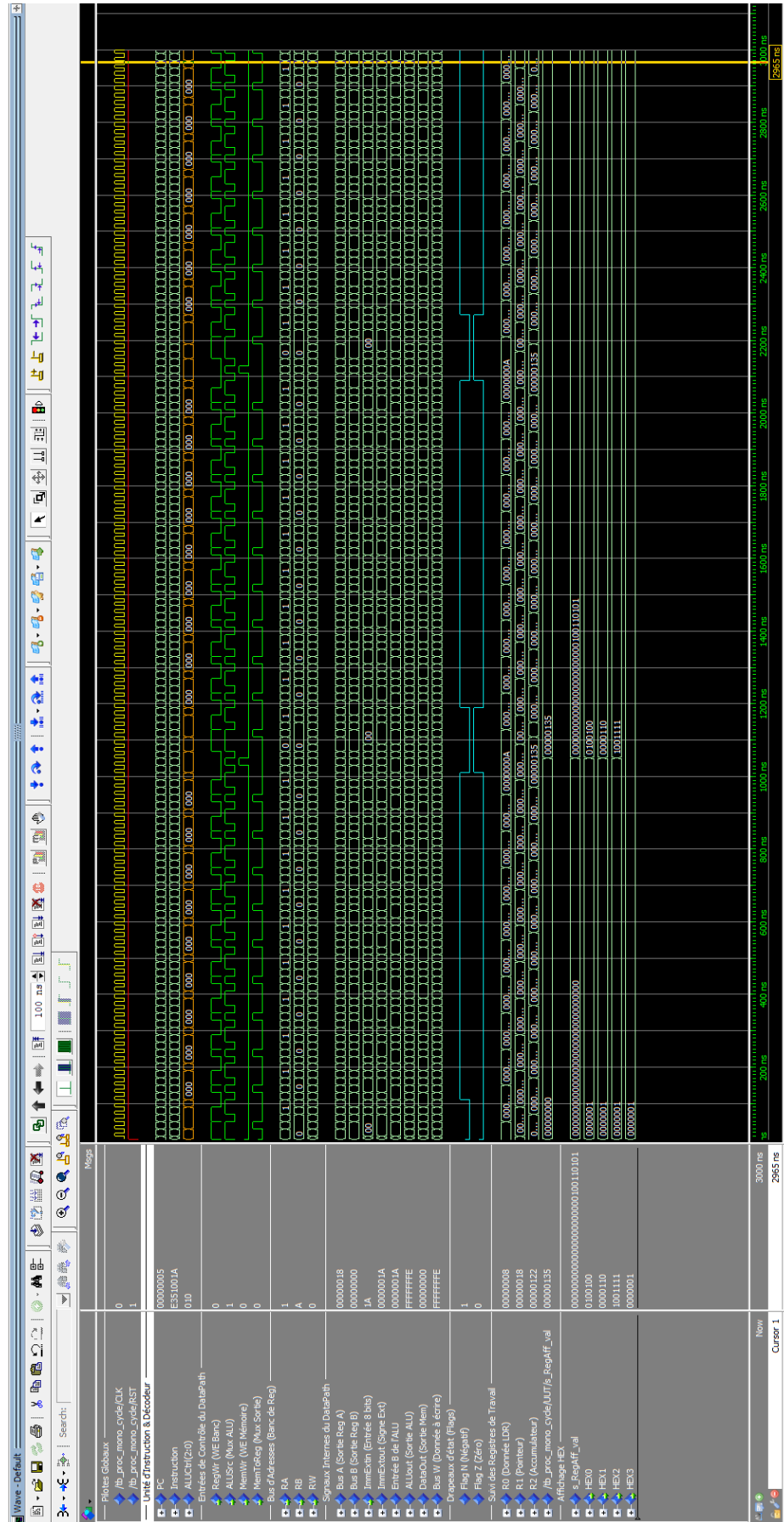


Figure 22 : Résultat de la simulation du monocycle

IV. Quartus prime

Pour la partie réalisée avec Quartus Prime, j'ai choisi de nommer le projet *mono_multi_cycle*.

Dans cette partie du projet, je me suis appuyé sur la documentation ainsi que sur la CAO électronique de la carte DE10-Lite afin d'identifier le raccordement des broches (pins).

L'une des problématiques rencontrées lors de ce projet avec Quartus Prime concernait le temps de compilation, qui variait entre 1 min 30 et 6 min. J'ai perdu beaucoup de temps sur cette partie en déboguant différentes erreurs. J'ai rapidement décidé de revenir à la simulation afin de vérifier correctement que les données affichées étaient bien les bonnes.

Je me suis rendu compte tardivement que j'avais en réalité mal configuré l'assignation des pins d'un des afficheurs.

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair	Strict Preservation
CLK	Input	PIN_P11	3	B3_NO	PIN_P11	3.3-V LVTTTL		8mA (default)			
HEX0[0]	Output	PIN_C14	7	B7_NO	PIN_C14	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX0[1]	Output	PIN_E15	7	B7_NO	PIN_E15	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX0[2]	Output	PIN_C15	7	B7_NO	PIN_C15	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX0[3]	Output	PIN_C16	7	B7_NO	PIN_C16	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX0[4]	Output	PIN_E16	7	B7_NO	PIN_E16	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX0[5]	Output	PIN_D17	7	B7_NO	PIN_D17	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX0[6]	Output	PIN_C17	7	B7_NO	PIN_C17	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[0]	Output	PIN_C18	7	B7_NO	PIN_C18	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[1]	Output	PIN_D18	6	B6_NO	PIN_D18	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[2]	Output	PIN_E18	6	B6_NO	PIN_E18	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[3]	Output	PIN_B16	7	B7_NO	PIN_B16	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[4]	Output	PIN_A17	7	B7_NO	PIN_A17	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[5]	Output	PIN_A18	7	B7_NO	PIN_A18	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[6]	Output	PIN_B17	7	B7_NO	PIN_B17	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[0]	Output	PIN_B20	6	B6_NO	PIN_B20	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[1]	Output	PIN_A20	7	B7_NO	PIN_A20	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[2]	Output	PIN_B19	7	B7_NO	PIN_B19	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[3]	Output	PIN_A21	6	B6_NO	PIN_A21	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[4]	Output	PIN_B21	6	B6_NO	PIN_B21	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[5]	Output	PIN_C22	6	B6_NO	PIN_C22	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[6]	Output	PIN_B22	6	B6_NO	PIN_B22	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[0]	Output	PIN_F21	6	B6_NO	PIN_F21	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[1]	Output	PIN_E22	6	B6_NO	PIN_E22	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[2]	Output	PIN_E21	6	B6_NO	PIN_E21	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[3]	Output	PIN_C19	7	B7_NO	PIN_C19	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[4]	Output	PIN_C20	6	B6_NO	PIN_C20	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[5]	Output	PIN_D19	6	B6_NO	PIN_D19	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[6]	Output	PIN_E17	6	B6_NO	PIN_E17	3.3-V LVTTTL		8mA (default)	2 (default)		
Pol_HEX	Input	PIN_C10	7	B7_NO	PIN_C10	3.3-V LVTTTL		8mA (default)			
RST	Input	PIN_B8	7	B7_NO	PIN_B8	3.3-V LVTTTL		8mA (default)			
<<new node>>											

Figure 23 : Pin planner du monocycle

A. Preuve de fonctionnement sur FPGA

Vous verrez dans les figures ci-dessous, les différentes preuves que le code fonctionne plutôt bien.

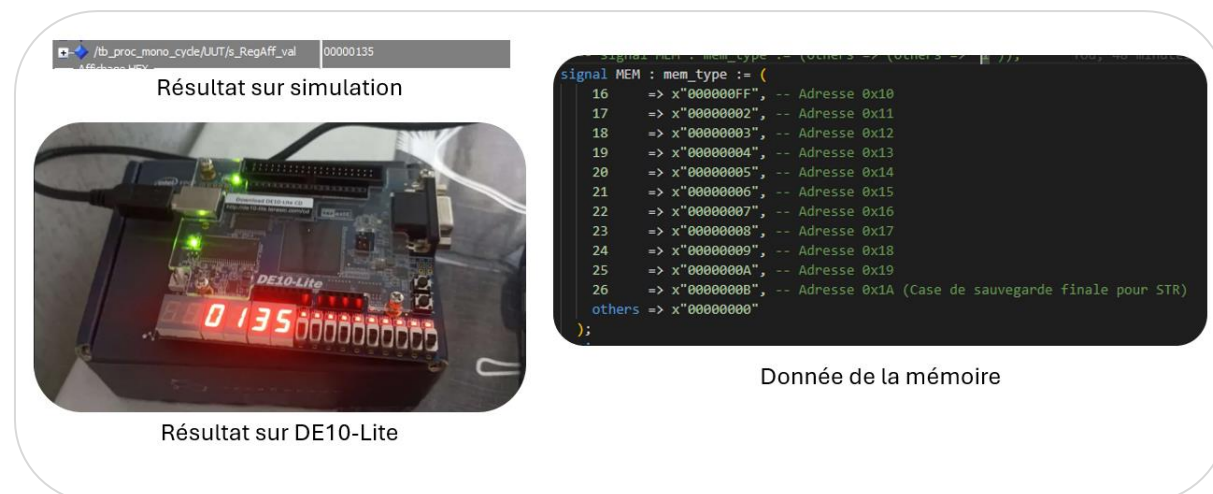


Figure 24 : Preuve de fonctionnement numéro 1

Pour la figure suivante, je me suis appuyé sur les résultats obtenus par Monsieur Dorian CAILLOU afin de comparer et de vérifier si j'avais obtenu les bons résultats. C'est bien le cas : pour les mêmes valeurs entrées en mémoire que celles utilisées dans son travail, j'obtiens les mêmes résultats.

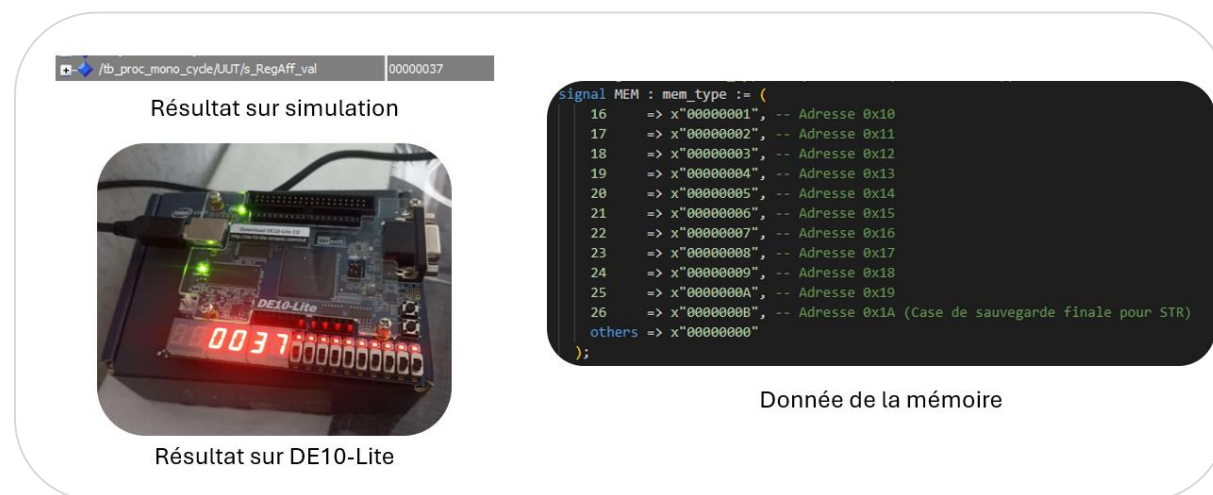


Figure 25 : Preuve de fonctionnement numéro 2

/tb_proc_mono_cycle/JUT/s_RegAff_val 000009F6

Résultat sur simulation



Résultat sur DE10-Lite

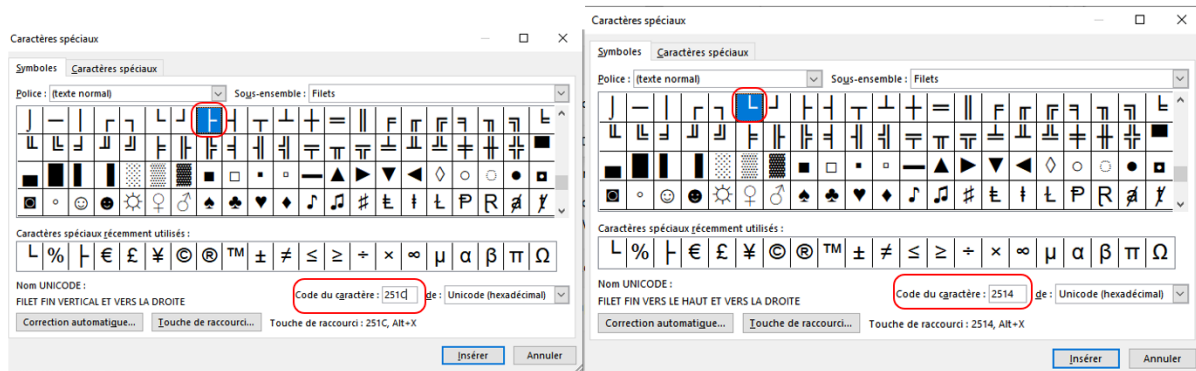
```
signal MEM : mem_type := (
16 => x"000000FF", -- Adresse 0x10
17 => x"000000FF", -- Adresse 0x11
18 => x"000000FF", -- Adresse 0x12
19 => x"000000FF", -- Adresse 0x13
20 => x"000000FF", -- Adresse 0x14
21 => x"000000FF", -- Adresse 0x15
22 => x"000000FF", -- Adresse 0x16
23 => x"000000FF", -- Adresse 0x17
24 => x"000000FF", -- Adresse 0x18
25 => x"000000FF", -- Adresse 0x19
26 => x"0000000B", -- Adresse 0x1A (Case de sauvegarde finale pour STR)
others => x"00000000"
);
```

Donnée de la mémoire

Figure 26 : Preuve de fonctionnement numéro 3

Annexe

Caractères spéciaux :



Tables des figures

Figure 1 : Carte DE10-Lite	7
Figure 2 : Représentation graphique de l'ALU	9
Figure 3 : Simulation ALU	11
Figure 4 : Représentation graphique du Banc de registre	11
Figure 5 : Simulation Datapath (Banc de registre + ALU)	13
Figure 6 : Représentation graphique du Multiplexeur 2 vers 1	14
Figure 7 : Simulation du Multiplexeur 2 vers 1	15
Figure 8 : Représentation graphique de l'Extension de signe	16
Figure 9 : Simulation de l'Extension du signe	17
Figure 10 : Représentation graphique de la Memory data	18
Figure 11 : Simulation de la Memory Data	20
Figure 12 : Représentation graphique du Datapath	20
Figure 13 : Assemblage des modules créer précédement.....	21
Figure 14 : Simulation du Datapaht	22
Figure 15 : Simulation du registre de commande 32 bits dans le datapath	23
Figure 16 : : Représentation graphique de l'Unité de Gestion d'Instruction	24
Figure 17 : Résultat de la simulation de l'Unité de Gestion d'Instruction	26
Figure 18 : Représentation graphique du Décodeur	27
Figure 19 : Valeur des commandes.....	27
Figure 20 : Résultat de la simulation du Decodeur	29
Figure 21 : Programme exécuté par le processeur monocycle.....	30
Figure 22 : Résultat de la simulation du monocycle	32
Figure 23 : Pin planner du monocycle	33
Figure 24 : Preuve de fonctionnement numéro 1	34
Figure 25 : Preuve de fonctionnement numéro 2.....	34
Figure 26 : Preuve de fonctionnement numéro 3.....	35